Project

Gestor de Finanzas Personales

Python Intermedio

Todo Entregas Discusión

¡Nada por ahora!

💼 Proyecto: Gestor de Finanzas Personales (con FreeSimpleGUI)

🎯 Objetivo

Desarrollar una **aplicación con interfaz gráfica** usando la biblioteca `FreeSimpleGUI`, que permita gestionar finanzas personales de forma sencilla. Este proyecto tiene como propósito poner en práctica **todos los conceptos aprendidos durante el Módulo 1**, especialmente:

Modularización

Manejo de archivos

COMIENZA EN 49 MIN
Resolución de dudas
Unirse

✅ Requisitos técnicos

La aplicación debe:

- Mostrar una **tabla de movimientos** (gastos e ingresos).
- Incluir un **botón para agregar una categoría**:
  - Al presionarlo, debe abrir una **nueva ventana** que permita ingresar una categoría (ej: Comida, Transporte, Entretenimiento...).
- Incluir un **botón para agregar un gasto**:
  - Al presionarlo, debe abrir una **nueva ventana** que permita ingresar un **título**, **monto** y **categoría** del gasto.
- Incluir un **botón para agregar un ingreso**:
  - Al presionarlo, debe abrir una **nueva ventana** que permita ingresar un **título**, **monto** y **categoría** del ingreso.
- Mostrar un **mensaje de error** si se intenta agregar un ingreso/gasto **sin categorías disponibles**.

💾 Persistencia de datos

- **Al cerrar o hacer un cambio**, el programa debe **guardar automáticamente** los datos en archivos.
- Al abrir el programa, debe **cargar los datos previamente guardados** (si existen).

🧠 Reglas de desarrollo

1. **Use clases** para representar las entidades principales del sistema (`Movimiento`, `Categoria`, `GestorFinanzas`, etc.).
2. **Separe la lógica del programa en funciones pequeñas y**

COMIENZA EN 49 MIN
Resolución de dudas
Unirse

Lyfter

4. Use **identificadores claros** que indiquen el propósito de cada variable, función o clase.
5. Aplique **validaciones básicas** en los formularios.
6. **Implemente pruebas unitarias** simples.

🧪 Pruebas unitarias

Se deben implementar **mínimo 8 pruebas unitarias.**

Las funciones o clases a probar **quedan a criterio del estudiante**, pero se recomienda enfocarse en la **lógica del programa y validaciones importantes**.

Las pruebas deben estar organizadas en un archivo aparte, por ejemplo: `test_logica.py`.

💡 Recuerde que las pruebas deben poder ejecutarse de forma independiente, sin depender de la interfaz gráfica.

🔍 Proceso sugerido

1. **Hacer mockups** de cada ventana.

   Investigar los elementos de `FreeSimpleGUI` necesarios.
2. **Diseñar la estructura de datos** en memoria (clases y relaciones).
3. **Dividir el desarrollo** en funciones/módulos probables por separado.
4. **Avanzar paso a paso**: no seguir al siguiente módulo sin verificar que el anterior funciona correctamente.
5. **Documente dudas** que le surjan durante el proceso.

Recomendaciones

COMIENZA EN 49 MIN
Resolución de dudas
Unirse

en la do

Lyfter

para conquistar).

4. Haga notas de aquellos requerimientos que no sepa cómo desarrollar con exactitud.
5. Divida la lógica del programa en funciones pequeñas y módulos.
6. Escriba código limpio. Use identificadores específicos y fáciles de entender.
7. Pregunte cualquier duda que tenga.

🔗 Recursos

FreeSimpleGUI en PyPI

Documentación oficial de PySimpleGUI

Requerimientos extra (opcionales)

1. Filtrar movimientos (gastos e ingresos) por un rango de fechas definido por el usuario

   Agregue dos campos (`Fecha inicio` y `Fecha fin`) en la ventana principal

   Al presionar un botón como “Filtrar”, la tabla debe actualizarse mostrando solo los movimientos dentro de ese rango

   Incluya validación del formato de fecha (`dd/mm/yyyy`)

   Ejemplo:

   Entrada:

```
[
    {"fecha": "02/07/2025", "título": "Sal
    {"fecha": "03/07/2025", "título": "Com
    {"fecha": "12/07/2025", "título": "Rop
]
```

COMIENZA EN 49 MIN
Resolución de dudas
Unirse

Salida:

Lyfter

```
3  - 02/07/2025 | Salario | ₡1000
4  - 03/07/2025 | Comida | ₡-20
```

2. Permitir al usuario definir la fecha del ingreso o gasto (en lugar de solo usar la actual)

   Al agregar un nuevo gasto o ingreso, incluya un campo adicional para ingresar la fecha (por defecto puede ser hoy)

   Valide que la fecha sea válida y que no esté en el futuro

   Ejemplo:

   Entrada:

```
{
    "título": "Venta libro",
    "monto": 50,
    "tipo": "Ingreso",
    "fecha": "20/07/2025"
}
```

   Salida correcta:

```
Nuevo ingreso agregado:
Fecha: 20/07/2025 | Título: Venta libro |
```

   Salida si la fecha es incorrecta:

```
Error: Formato de fecha inválido (use dd/m
```

   Salida si la fecha es futura:

```
Error: La fecha no puede ser en el futuro
```

COMIENZA EN 49 MIN
Resolución de dudas
Unirse

es

pal

Lyfter

`Tipo`

Suma total de ingresos, gastos, y balance neto al final

Ejemplo:

Entrada:

```
[
    {"fecha": "01/07/2025", "título": "Sal
    {"fecha": "02/07/2025", "título": "Com
]
```

Salida del CSV:

Presentar Sin acción

```

Totales:
Ingresos: ₡1200
Gastos: ₡100
Balance Neto: ₡1100
```

4. Asignar un color personalizado a cada categoría

   Al crear o editar una categoría, permita seleccionar un color con `sg.ColorChooserButton`

   Al mostrar los movimientos en la tabla, use el color asociado a su categoría

   Asegúrese de guardar esta información junto con las categorías

   Ejemplo:

   Entrada:

```
Categoría: "Comida"
```

COMIENZA EN 49 MIN
Resolución de dudas
Unirse

Lyfter

Salida:

En la tabla del GUI, la fila de "Pizza" aparece con color de fondo `#FFA500` (naranja)

A nivel de base de datos:

```
{
    "categorías": {
        "Comida": "#FFA500"
    }
}
```

COMIENZA EN 49 MIN
Resolución de dudas
Unirse